

CustomerPilot V6.2

Enterprise Hardening Addendum

12 enterprise reliability points from the 3rd audit.

Documentation-only. No new features. Production governance for scale.

Concise decision-matrix format

V6.2 Business Rule Fixes (Applied)

Fix #1: Owner Override Inventory

Gift rewards now decrement `Reward.stock` - (pastry physically consumed). Blocks if `stock=0`.
Before: Stock NOT decremented (V6.1 policy said 'gift, not inventory'). *After:* Physically consumed = inventory decremented. Audit's correction applied.

Fix #2: Fraud Severity Matrix

4 levels: info | warning | high | critical. `shouldRequireManagerApproval()` triggers on 2+ HIGH or 1+ CRITICAL in 24h -> `Staff.requireManagerApprovalUntil` set for 24h. While set, cashier's bills require manager approval before stamps awarded.

Audit's final verdict: "ab sabse valuable validation real pilot, QA automation, aur production telemetry se hi milegi, documentation se nahi." The 12 points below define production behavior for when CustomerPilot scales beyond single-merchant demo. They are documentation-only in V6.2 - no enforcement code added per audit's explicit guidance.

Table of Contents

1.	Idempotency for External Webhooks Razorpay/Google/WhatsApp webhooks can retry or duplicate. How do we prevent double-processing?	Critical
2.	Concurrency & Distributed Locking Rule change + rollback + customer redeem + stock update simultaneously. Race conditions?	Critical
3.	Cron Retry & Dead-Letter Queue 7+ cron jobs. What happens when they fail? Partial completion? Timeout? Retry storm?	Critical
4.	Multi-Device Owner Session Management Owner logs in from desktop + mobile + tablet simultaneously. Active sessions? Force logout?	High
5.	WhatsApp Permanent Failure Handling WhatsApp permanently fails (invalid number, template rejected, customer opted out). What happens to queue?	High
6.	Merchant Support Ticket Lifecycle Support tickets need SLA, priority, reopening, escalation. Current model is basic.	High
7.	Backup, Restore & Disaster Recovery Production data loss (DB corruption, accidental delete, cloud outage). Recovery strategy?	High
8.	Merchant Notification Preferences Merchant gets daily summary, morning briefing, fraud alerts, birthday/win-back digest. Can they customize?	Medium
9.	Data Export Policy Merchant says 'Give me all my data in Excel'. Customers ask 'Give me my data' (GDPR/DPDP). What's exportable?	Medium
10.	Retention Policy for Soft-Deleted Entities Customer merge soft-deletes (30-day hard delete). What about Staff, Rewards, Bills, AuditLog, WhatsApp?	Medium
11.	API Versioning Strategy V6 has /api/bills. Future V7 might change response schema. How do we version without breaking merchants?	Medium
12.	Feature Licensing Matrix Starter / Pro / Enterprise plans. What's in each? Merchant needs to see upgrade path.	Medium

1

Idempotency for External Webhooks

Razorpay/Google/WhatsApp webhooks can retry or duplicate. How do we prevent double-processing?

Decision Matrix

■ Trust webhooks — process every callback as new

Simplest. But double-charges, duplicate redemptions on retry.

✓ Idempotency keys: store event_id, skip if already processed

Standard pattern. Each event gets unique ID. Process once.

RECOMMENDED

■ Time-window dedup: ignore same-event-type within 5 min

Fragile. Legitimate rapid events dropped.

Production Behavior

Every external webhook MUST include event_id (provider-supplied). On receipt: (1) check WebhookEventLog for event_id, (2) if exists -> return 200 OK silently, (3) if not -> process + insert event_id. Schema: { id, provider, event_id, received_at, processed_at, status, payload_hash }. Retention: 90 days. Unique index on (provider, event_id).

Audit Actions

WEBHOOK_RECEIVED | WEBHOOK_PROCESSED | WEBHOOK_DUPLICATE_IGNORED |
WEBHOOK_PROCESSING_FAILED

Implementation:

2

Concurrency & Distributed Locking

Rule change + rollback + customer redeem + stock update simultaneously. Race conditions?

Decision Matrix

■ Single-threaded DB operations — rely on serializability

Works for SQLite demo. Fails at scale with concurrent merchants.

✓ Optimistic locking: version column, retry on conflict

Standard. Add version INT to critical tables. WHERE version = X. If 0 rows -> retry 3x.

RECOMMENDED

■ Pessimistic locking: SELECT FOR UPDATE on every write

Safe but slow. Deadlocks possible. Overkill.

Production Behavior

Optimistic locking on: Customer, CustomerStampCard, Reward, Bill, Redemption. Each has `version INT DEFAULT 0`. On update: UPDATE ... SET version = version + 1 WHERE id = ? AND version = ?. If 0 rows -> conflict -> retry 3x with 100ms backoff. After 3 retries -> 409 Conflict. Client shows 'Concurrent modification - refresh and retry'. Critical operations (rule change, ownership transfer) use SERIALIZABLE transactions.

Audit Actions

OPTIMISTIC_LOCK_RETRY | OPTIMISTIC_LOCK_CONFLICT_FAILED |
TRANSACTION_SERIALIZABLE_USED

Implementation:

Decision Matrix

■ Fire-and-forget — wait for next run

Missed birthdays, stuck waitlists, undelivered summaries. Silent failures.

✓ Retry with exponential backoff + dead-letter queue

Standard. 3 retries (1s, 10s, 60s). If still failing -> DLQ. Alert ops.

RECOMMENDED

■ Circuit breaker — pause after 5 consecutive failures

Prevents retry storms. But may pause legitimate cron.

Production Behavior

Every cron wrapped in CronJobRunner. On failure: retry 3x with exponential backoff (1s -> 10s -> 60s). If still failing -> insert into CronDeadLetterQueue { jobName, payload, lastError, failedAt, retryCount }. Alert ops + Slack webhook. DLQ reviewed daily, can be manually re-queued. Auto-purge after 7 days. Job timeout: 5 min max. Partial completion: each job is idempotent (safe to re-run).

Audit Actions

CRON_JOB_STARTED | CRON_JOB_SUCCEEDED | CRON_JOB_FAILED |
CRON_JOB_RETRYING | CRON_JOB_DLQ_INSERTED | CRON_JOB_TIMEOUT

Implementation:

4

Multi-Device Owner Session Management

Owner logs in from desktop + mobile + tablet simultaneously. Active sessions? Force logout?

Decision Matrix

■ Single session — new login auto-logs-out old

Strict. Can't use phone + desktop. Frustrating.

■ Unlimited sessions, no management

Convenient. But security risk — lost device = permanent access.

✓ Up to 5 concurrent sessions, revocable from dashboard

Balanced. Owner sees sessions, can revoke. Beyond 5 -> oldest revoked.

RECOMMENDED

Production Behavior

StaffSession table: { id, staffId, deviceInfo, ipAddress, createdAt, lastActiveAt, revokedAt }. On login: create session. If active >= 5 -> revoke oldest. Dashboard (Settings -> Active Sessions): device list, last-active, 'Revoke' per session. 'Revoke all' for emergency. Session expires after 30 days inactivity. On password change: all sessions revoked. On staff suspended: all revoked. JWT includes sessionId for validation on every API call.

Audit Actions

SESSION_CREATED | SESSION_REVOKED | SESSION_EXPIRED | SESSION_LIMIT_REACHED | ALL_SESSIONS_REVOKED

Implementation:

5

WhatsApp Permanent Failure Handling

WhatsApp permanently fails (invalid number, template rejected, customer opted out). What happens to queue?

Decision Matrix

■ Retry forever — never give up

Spams provider. Wastes quota. Annoys ops with stuck queue.

■ 3 retries then 'failed', no recovery

Simple. But customer never gets message. No opt-out recovery.

✓ 3 retries -> classify -> opt-out recovery for soft, permanent block for hard

Comprehensive. Soft (timeout) auto-retry next day. Hard (invalid number) -> whatsappOptIn=false.

RECOMMENDED

Production Behavior

After 3 retries, classify: SOFT (timeout, rate limit) -> status='failed_soft', retry once more after 24h. HARD (invalid number, template rejected, blocked) -> status='failed_hard', customer.whatsappOptIn=false, customer notified at next POS visit ('We couldn't reach you on WhatsApp'). Template rejected -> alert ops, template disabled. Opt-out recovery: customer texts 'START' -> whatsappOptIn=true, queue resumes. Permanent block: only on customer request or 10+ hard failures.

Audit Actions

WHATSAPP_FAILED_SOFT | WHATSAPP_FAILED_HARD | WHATSAPP_OPT_OUT_TRIGGERED |
WHATSAPP_OPT_IN_RECOVERED | WHATSAPP_TEMPLATE_REJECTED |
WHATSAPP_PERMANENT_BLOCK

Implementation:

Merchant Support Ticket Lifecycle

Support tickets need SLA, priority, reopening, escalation. Current model is basic.

Decision Matrix

■ Basic open/resolved states only

Current V6.1. No SLA tracking. No escalation. Tickets get lost.

✓ Full ITSM lifecycle + SLA + reopening

Standard support workflow. SLA per priority. Auto-escalation on breach.

RECOMMENDED

■ AI-triaged: LLM categorizes + prioritizes

Advanced. But adds LLM dependency. Defer to V7.

Production Behavior

States: open -> in_progress -> waiting_customer -> resolved -> closed. SLA by priority: urgent=4h, high=24h, normal=72h, low=168h. SLA timer pauses while waiting_customer. On breach: auto-escalate priority + alert ops. Resolved tickets auto-close after 7 days if no reopen. Reopen within 7 days -> priority +1. After 7 days: closed permanently. Dashboard: 'Tickets breaching SLA' widget. Merchant rates resolution (helpful/not helpful).

Audit Actions

TICKET_CREATED | TICKET_PRIORITY_ESCALATED | TICKET_SLA_BREACHED |
TICKET_REOPENED | TICKET_CLOSED | TICKET_FEEDBACK_RECEIVED

Implementation:

Decision Matrix

■ Rely on cloud provider backups only

Simple. But limited restore granularity. No tested procedure.

✓ Daily snapshots + weekly restore drills + documented RTO/RPO

Industry standard. RPO=24h, RTO=4h. Tested quarterly.

RECOMMENDED

■ Real-time replication + cross-region failover

Enterprise-grade. RPO=0, RTO<15min. But expensive. Overkill for V6.2.

Production Behavior

Backup: daily full DB snapshot at 3 AM IST + continuous WAL streaming. Retention: 30 daily + 12 monthly + 7 yearly. Storage: encrypted, cross-region (Mumbai + Singapore). Restore: quarterly drill - restore to staging, verify integrity, document time-to-restore. RPO=24h (max 1 day loss). RTO=4h (max 4h downtime). DR runbook: (1) provision new DB from snapshot, (2) update connection string, (3) verify counts, (4) switch DNS, (5) post-incident report. Backup monitoring: alert if backup fails or snapshot size drops >50%.

Audit Actions

BACKUP_STARTED | BACKUP_COMPLETED | BACKUP_FAILED | RESTORE_DRILL_COMPLETED
| RESTORE_DRILL_FAILED | DISASTER_RECOVERY_TRIGGERED

Implementation:

Merchant Notification Preferences

Merchant gets daily summary, morning briefing, fraud alerts, birthday/win-back digest. Can they customize?

Decision Matrix

■ All-or-nothing - receive every notification

Simple. But notification fatigue. Owner may mute entirely.

✓ Per-type toggle + per-channel (WhatsApp/Email/Dashboard)

Maximum control. 5 types x 3 channels = 15 toggles.

RECOMMENDED

■ Quiet hours - no notifications 10 PM - 8 AM

Reduces annoyance but doesn't solve relevance.

Production Behavior

MerchantNotificationPreference table: { merchantId, notificationType, channel, enabled }. Types: daily_summary, morning_briefing, fraud_alert, birthday_digest, win_back_digest, review_request, low_stock_alert, subscription_alert. Channels: whatsapp, email, dashboard. Defaults by role: Owner=all, Manager=operational, Cashier=none. Dashboard always enabled. Urgent alerts (fraud critical, subscription failed) bypass preferences. Quiet hours: configurable (default 10 PM - 7 AM, urgent still sent).

Audit Actions

NOTIFICATION_PREFERENCE_CHANGED | NOTIFICATION_SENT |
NOTIFICATION_SUPPRESSED_BY_PREFERENCE | NOTIFICATION_URGENT_BYPASS

Implementation:

Data Export Policy

Merchant says 'Give me all my data in Excel'. Customers ask 'Give me my data' (GDPR/DPDP). What's exportable?

Decision Matrix

■ No export - data locked in system

Vendor lock-in. Violates DPDP Act. Merchants leave.

■ Manual export by support on request

Slow (3-5 days). Doesn't scale. Support bottleneck.

✓ Self-service export from dashboard, CSV/Excel, async generation

Standard SaaS. Merchant clicks 'Export', gets email link in 5-30 min.

RECOMMENDED

Production Behavior

Settings -> Data Export. Merchant picks: entity type (Customers/Bills/Redemptions/Reviews/AuditLog/All), date range, format (CSV/Excel/JSON). Async job -> DataExportJob { merchantId, entities, dateRange, format, status, downloadUrl, expiresAt }. Background worker (5-30 min). On completion: email + WhatsApp with secure signed URL (7-day expiry). Customer self-export: JSON with their stamps, bills, redemptions, reviews. Privacy: AuditLog tracks who exported what. Rate limit: 1 export per merchant per hour. Large exports (>100K rows) chunked into ZIP.

Audit Actions

DATA_EXPORT_REQUESTED | DATA_EXPORT_COMPLETED | DATA_EXPORT_FAILED |
DATA_EXPORT_DOWNLOADED | CUSTOMER_DATA_EXPORTED

Implementation:

Retention Policy for Soft-Deleted Entities

Customer merge soft-deletes (30-day hard delete). What about Staff, Rewards, Bills, AuditLog, WhatsApp?

Decision Matrix

■ Keep everything forever

Storage grows unbounded. GDPR/DPDP violations.

■ Hard delete immediately on soft-delete

No recovery window. Accidental deletes permanent.

✓ Tiered retention: 30-90-365 days by entity type, then hard delete

Compliant + practical. Recovery windows per criticality.

RECOMMENDED

Production Behavior

Retention schedule: Customer (merge soft-delete): 30 days. Customer (requested deletion): 30 days cold storage (DPDP). Staff (suspended/resigned): 90 days. Rewards (inactive): 90 days (after pending redemptions resolved). Bills (voided/refunded): 365 days. Bills (confirmed): 7 years (GST law). AuditLog: 7 years (compliance). WhatsAppMessage: 90 days. Reviews: 365 days after clawback/expiry. Cron: retention-enforcer.ts daily. Hard-deletes past retention. AuditLog: RETENTION_HARD_DELETE per batch.

Audit Actions

RETENTION_SOFT_DELETE | RETENTION_HARD_DELETE |
RETENTION_RECOVERY_WINDOW_EXPIRED

Implementation:

API Versioning Strategy

V6 has /api/bills. Future V7 might change response schema. How do we version without breaking merchants?

Decision Matrix

■ No versioning - change API, break clients

Simplest. But merchants' integrated systems break. Trust lost.

✓ URL versioning: /v1/api/bills, /v2/api/bills

Industry standard. Clear, cacheable, explicit. Old versions 12-month sunset.

RECOMMENDED

■ Header versioning: Accept-Version: v1

Clean URLs. But invisible in logs, harder to debug.

Production Behavior

URL versioning: /api/v1/bills, /api/v2/bills. Default (no version) -> latest stable. Lifecycle: v1 released -> v2 released (v1 deprecated, 12-month sunset) -> v1 sunset (410 Gone). Breaking changes (response schema, required params, status codes) require new version. Non-breaking (new optional params, new response fields) backward-compatible. Deprecation headers: API-Deprecation: true, Sunset: . Developer Manual documents per-version. Current: v1 (V6.2). Next: v2 when multi-tenant auth lands.

Audit Actions

API_VERSION_DEPRECATED | API_VERSION_SUNSET | API_BREAKING_CHANGE INTRODUCED

Implementation:

Feature Licensing Matrix

Starter / Pro / Enterprise plans. What's in each? Merchant needs to see upgrade path.

Decision Matrix

■ Single plan - all features for everyone

Simple. But no upsell path. Power users and small merchants pay same.

✓ 3-tier matrix: Starter / Pro / Enterprise, feature-gated

Standard SaaS. Clear value ladder. Locked features show 'Upgrade'.

RECOMMENDED

■ Usage-based pricing - pay per bill/customer/stamp

Fair but unpredictable. Hard to forecast revenue.

Production Behavior

Plans: Starter (Rs 999/mo) - 1 staff, 100 customers, 1 stamp card, basic reports, WhatsApp 100/mo. Pro (Rs 2,499/mo) - 5 staff, 1000 customers, 5 stamp cards, advanced reports + AI insights, WhatsApp 1000/mo, VIP tiers, birthday engine, review engine. Enterprise (Rs 9,999/mo) - unlimited, white-label, multi-branch, API access, priority support. Feature gating: each feature checks merchant.plan. Locked features show 'Upgrade to Pro' CTA. Over-limit: soft warnings at 80%, hard block at 100%. Upgrade: pay prorated, features unlock instantly. Downgrade: next billing cycle (no mid-cycle loss).

Audit Actions

PLAN_UPGRADED | PLAN_DOWNGRADED | FEATURE_LIMIT_REACHED |
FEATURE_LIMIT_BLOCKED | PLAN_PAYMENT_FAILED

Implementation: